

UNIVERSITY OF ZAGREB
FACULTY OF ELECTRICAL ENGINEERING AND COMPUTING

BACHELOR THESIS No. 1433

**EXAMINING GRAPH ISOMORPHISM USING DEEP GRAPH
NEURAL NETWORKS**

Anabel Dautović

Zagreb, June 2024

UNIVERSITY OF ZAGREB
FACULTY OF ELECTRICAL ENGINEERING AND COMPUTING

BACHELOR THESIS No. 1433

**EXAMINING GRAPH ISOMORPHISM USING DEEP GRAPH
NEURAL NETWORKS**

Anabel Dautović

Zagreb, June 2024

BACHELOR THESIS ASSIGNMENT No. 1433

Student: **Anabel Dautović (0036541492)**
Study: Electrical Engineering and Information Technology and Computing
Module: Computing
Mentor: assoc. prof. Anamari Nakić

Title: **Examining graph isomorphism using deep graph neural networks**

Description:

A graph is a combinatorial structure, it is a pair $G = (V, E)$ of a non-empty finite set of vertices V and a finite set E of 2-subsets of V , which we call edges. Graph theory, a branch of mathematics that studies problems related to graphs, has been developing rapidly in the last fifty years due to numerous applications in modeling and analyzing networks. It is a challenging problem to recognize whether two networks are equal, i.e. whether the corresponding abstract graph models are isomorphic. Graphs G and H are said to be isomorphic if there exists a bijective correspondence between the sets $V(G)$ and $V(H)$, such that the number of edges connecting any two vertices in $V(G)$ is equal to the number of edges connecting the corresponding two vertices in $V(H)$. The problem of determining the isomorphism of two graphs is a difficult computational problem of exponential complexity. Therefore, it is appropriate to investigate the possibilities of solving this problem using deep graph neural networks. Such neural networks consist of multiple layers of neurons and are used in deep learning to learn complex and abstract representations of data. Graph neural networks are a type of neural network designed to work with data structured in the form of graphs. In this thesis, a deep graph neural network that is trained to predict whether two graphs are isomorphic will be built. A dataset will be generated for testing and validation. The influence of different network architectures and other parameters on the success of network prediction will be visualized. Different results depending on the initial parameters will be compared and analyzed.

Submission date: 14 June 2024

ZAVRŠNI ZADATAK br. 1433

Pristupnica: **Anabel Dautović (0036541492)**
Studij: Elektrotehnika i informacijska tehnologija i Računarstvo
Modul: Računarstvo
Mentorica: izv. prof. dr. sc. Anamari Nakić

Zadatak: **Ispitivanje izomorfности grafova s pomoću dubokih graf neuronskih mreža**

Opis zadatka:

Graf je kombinatorna struktura, to je uređeni par $G = (V, E)$ nepraznog konačnog skupa vrhova V i konačnog skupa E dvočlanih podskupova od V koje nazivamo bridovima. Teorija grafova, grana matematike koja izučava probleme vezane uz pojam grafa, ubrzano se razvija u posljednjih pedesetak godina zbog brojnih primjena u modeliranju i analiziranju mreža. Izazovan je problem prepoznati jesu li dvije mreže jednake, odnosno jesu li pripadajući apstraktni modeli grafa izomorfni. Za grafove G i H kažemo da su izomorfni ako postoji bijektivna korespondencija između skupova $V(G)$ i $V(H)$, takva da je broj bridova koji spajaju bilo koja dva izabrana vrha u $V(G)$ jednak broju bridova koji spajaju korespondentna dva vrha u $V(H)$. Problem određivanja izomorfности dva grafa je težak računalni problem eksponencijalne složenosti. Stoga je primjereno istražiti mogućnosti rješavanja ovoga problema s pomoću dubokih graf neuronskih mreža. Takve se neuronske mreže sastoje od više slojeva neurona, a koriste se u dubokom učenju kako bi naučile složene i apstraktne reprezentacije podataka. Graf neuronske mreže su vrsta neuronskih mreža dizajnirane za rad s podacima strukturiranim u obliku grafova. U ovom radu izradit će se duboka graf neuronska mreža koja je istrenirana da predviđa jesu li dva grafa izomorfna. Generirat će se skup podataka za testiranje i validaciju. Vizualizirat će se utjecaj različitih arhitektura mreže i drugih parametara na uspješnost predikcije mreže. Usporedit će se i analizirati različiti rezultati u ovisnosti o početnim parametrima.

Rok za predaju rada: 14. lipnja 2024.

Contents

1	Introduction	3
2	Introduction to Graph Theory	4
2.1	Graph Theory Fundamentals	4
2.2	Graph Isomorphism	5
3	Deep Neural Networks	8
3.1	Activation Functions	9
3.2	Optimization Functions	10
3.3	Error Functions	11
3.4	Graph Neural Networks	11
3.5	Types of Graph Neural Networks	12
3.6	GNNs in the context of graph isomorphism problem, their power and limitations	15
3.7	Representation of Graphs	15
3.8	Injectivity	15
3.9	Expressiveness	16
3.10	Oversmoothing	16
4	Data generation	17
5	Implementation	20
5.1	Models	20
5.2	Training and Evaluation	21
6	Experiments	22
6.1	Results	22

6.1.1	GCN	23
6.1.2	GraphConv	23
6.1.3	SAGE	24
6.1.4	GAT	24
6.1.5	ChebConv	25
6.1.6	ARMAConv	25
6.1.7	SGConv	26
6.1.8	GINConv	26
6.2	Comparison	27
7	Discussion	31
8	Conclusion	32
9	Code	33
	References	34

1 Introduction

Graph isomorphism testing stands as a fundamental problem in computational sciences, underpinning various domains such as chemistry, network analysis, and cybersecurity. Graph Neural Networks (GNNs) have recently emerged as a promising approach for addressing the complexities of isomorphism testing, leveraging their ability to capture intricate relationships and structural dependencies within graphs.

The ability of GNNs to determine graph isomorphism is closely tied to their architecture and the mechanisms they use to aggregate and update node features. Different GNN architectures, such as GCN (Graph Convolutional Network), SAGE (GraphSAGE), GAT (Graph Attention Network), GraphConv (Graph Convolution), ChebConv (Chebyshev Convolution), ARMAConv (Auto-Regressive Moving Average Convolution), SGConv (Spline-based Convolution) and GINConv (Graph Isomorphism Network), vary in their expressive power and ability to distinguish non-isomorphic graphs. This thesis compares the performance of different GNN architectures in determining graph isomorphism by evaluating them on a specially generated dataset, aiming to assess their effectiveness in distinguishing between isomorphic and non-isomorphic graph pairs. Through empirical analysis, it provides insights into the efficacy of various widely-used Graph Neural Networks (GNNs) in aggregating node information and capturing structural nuances crucial for accurate isomorphism detection, thus revealing the relative strengths and limitations of different GNN approaches for graph isomorphism testing.

2 Introduction to Graph Theory

2.1 Graph Theory Fundamentals

Here are some most important definitions necessary for this thesis.

A **Graph** G is represented as an ordered pair $G = (V, E)$, where V is the set of vertices (nodes) and E is the set of edges (links) connecting pairs of vertices.

Directed Graph is a graph where each edge is associated with a direction, indicating a one-way relationship between vertices. Mathematically, an edge (u, v) in a directed graph goes from vertex u to vertex v .

Undirected Graph is a graph where edges do not have a direction, implying a symmetric relationship between vertices. Mathematically, an edge $\{u, v\}$ in an undirected graph connects vertices u and v without any specified direction.

Weighted Graph is a graph where each edge is assigned a numerical value, known as a weight, representing a cost or distance associated with traversing that edge.

In an undirected graph $G = (V, E)$, the **degree** $d(v)$ of a node v is the number of edges incident to it. Mathematically, the degree of node v is defined as:

$$d(v) = |\{(u, v) \in E \mid u \in V\}|$$

In a directed graph, each node v has an *input degree* $d_{\text{in}}(v)$, representing the number of edges pointing towards it, and an *output degree* $d_{\text{out}}(v)$, indicating the number of edges originating from it.

Adjacency Matrices A is a square matrix used to represent the connections between

vertices in a graph. For a graph with n vertices, the adjacency matrix A is an $n \times n$ matrix defined as:

$$A[i][j] = \begin{cases} 1, & \text{if there is an edge between vertices } i \text{ and } j \\ 0, & \text{otherwise} \end{cases}$$

2.2 Graph Isomorphism

Two graphs G_1 and G_2 are isomorphic if there exists a bijective function f that maps each node of G_1 to a node of G_2 such that, the number of edges connecting any two nodes x and y in G_1 is equal to the number of edges connecting the nodes $f(x)$ and $f(y)$ in G_2 [1]. Put simply, if two graphs G_1 and G_2 can be redrawn to give the same picture, they are considered isomorphic. The question whether an algorithm can determine the graph isomorphism in polynomial time remains a fundamental and unresolved problem in computer science, referred to as the graph isomorphism problem.

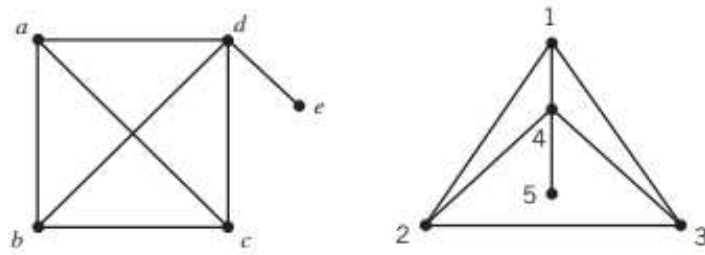


Figure 2.1: Example of isomorphic graphs [2]

The complexity of the graph isomorphism problem stems from the need to consider every possible mapping of vertices between two graphs. As the number of vertices increases, the number of potential mappings grows exponentially, making it computationally intensive to verify isomorphism for large graphs. While there is no formula to count the number of graphs for n number of vertices, the number of non-isomorphic graphs grows quickly, surpassing a billion for graphs with 11 vertices [3]. This computational difficulty is compounded by the fact that there are no known efficient algorithms that can solve the problem for all graphs.

Furthermore, the problem is one of the few problems in the NP class that is neither proven to be solvable in polynomial time (P) nor proven to be NP-complete. It's consid-

ered GI-complete, GI standing for Graph Isomorphism, with the best known algorithms, specifically, Babai's algorithm [4], having quasi-polynomial time complexity.

In practice, the most straightforward approach for determining isomorphism is using brute force, generating all possible permutations of one graph's vertices and checking if the resulting graph is isomorphic to the other. This approach is impractical for large graphs due to its exponential time complexity. Other algorithms used to tackle the problem include backtracking algorithms, the Weisfeiler-Lehman (WL) algorithm [5], and Nauty and Traces [6].

Backtracking algorithms explore the search space of possible mappings between the vertices of two graphs. One such algorithm is the **Ullmann algorithm** [7], which incrementally extends a partial mapping and backtracks when necessary. Backtracking algorithms, like Ullmann's, are limited for graph isomorphism testing due to their exponential time complexity, hindering their practicality for large graphs. They also encounter scalability issues, struggle with highly symmetrical graphs, and require substantial memory, restricting their use to smaller graphs where computational challenges are more manageable.

The **Weisfeiler-Lehman (WL) algorithm** is a heuristic algorithm that uses the concept of vertex color refinement. The key idea is to iteratively refine a color assignment based on the local structure of the graph until a fixed point is reached. Initially, each vertex is assigned a color (or label) based on its degree. In each iteration, the color of each vertex is updated based on the multiset of colors in its neighborhood. This process continues until the color distribution stabilizes. The WL algorithm, while efficient and widely used for graph isomorphism testing, has notable limitations. It cannot distinguish certain non-isomorphic graphs known as WL-equivalent graphs, which have identical color refinement sequences. The algorithm's focus on local structure often misses global structural differences, and it struggles with highly symmetrical graphs. Additionally, its heuristic nature means it is not guaranteed to work in all cases, and its computational performance can degrade with large graphs.

Nauty is a program designed for isomorphism testing and graph canonicalization, aiming to find a standardized representation for a graph. It combines backtracking tech-

niques and group theory to prune the search space efficiently. **Traces** complements Nauty by leveraging similar principles, yet it's tailored for specific graph classes, which may result in optimized performance for certain scenarios but potentially less efficiency for others.

3 Deep Neural Networks

Deep neural networks are neural networks consisting of multiple layers of neurons and are used in deep learning to learn complex and abstract data representations. The basic building block of a deep neural network is a neuron, which is a simple processing node with inputs, weights, bias, and an activation function. Multiple layers of neurons interconnected form a deep neural network. The key challenge in deep learning is to find the optimal function that maps input data (x) to corresponding output data (Y). A neuron operates by computing the sum of input data weighted by parameters (W) and adding bias (b).

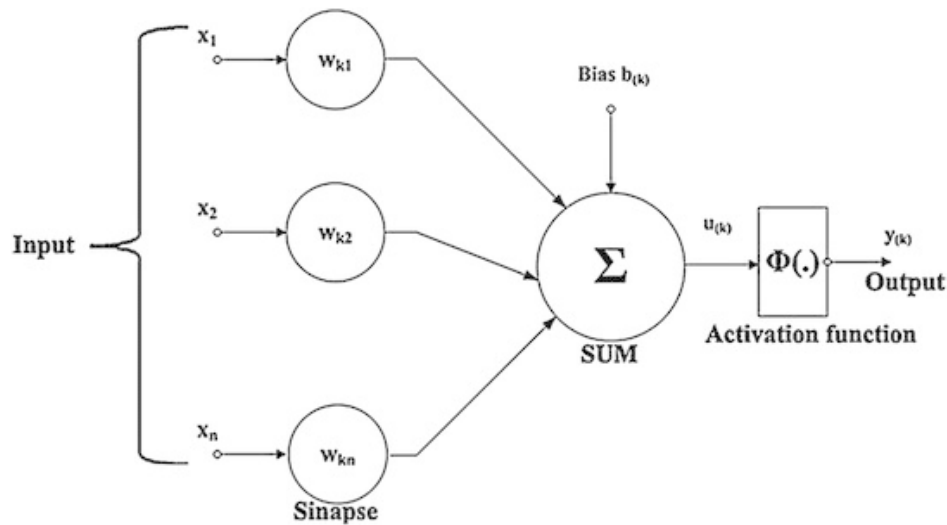


Figure 3.1: Artificial neuron [8]

Deep neural networks often use batch learning, where updates to the network's parameters are made using batches of training data rather than individual data points. This makes learning more efficient and stable, using parallel computing and patterns in data.

Deep neural networks are organized in layers: input, hidden, and output. The input layer receives data, hidden layers process information, while the output layer provides

final prediction results.

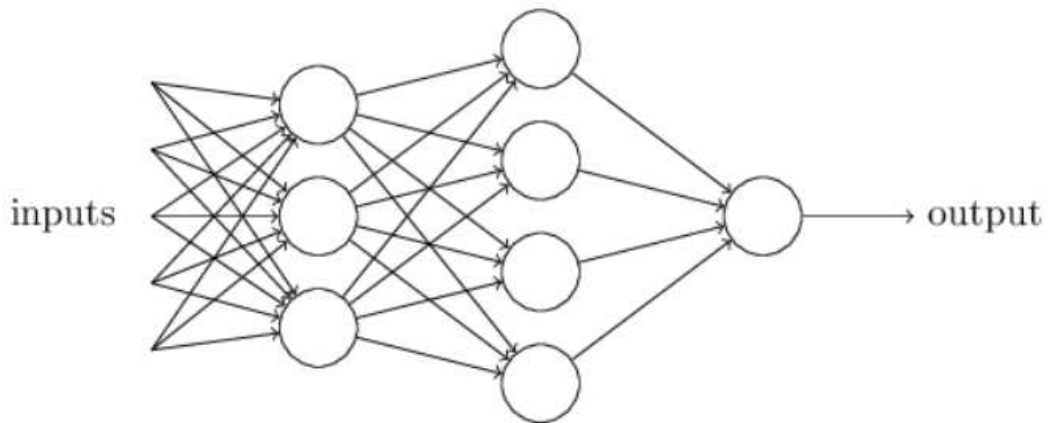


Figure 3.2: Representation of input and output layers [9]

In deep learning, the selection of appropriate activation functions, optimization algorithms, and loss functions plays a crucial role in successfully training neural networks.

3.1 Activation Functions

Activation functions add nonlinearity to artificial neural networks, and some commonly used functions include the sigmoid function, hyperbolic tangent, rectified linear unit, and softmax function.

Sigmoid Function

Often used in binary classifications as it maps input to values between 0 and 1, which can be interpreted as probabilities.

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

Hyperbolic Tangent (tanh)

Similar to the sigmoid function but maps input values between -1 and 1. Often used in hidden layers of neural networks.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

Rectified Linear Unit (ReLU)

For positive input values, it returns the same value, while for negative input, it returns zero. Helps address the vanishing gradient problem.

$$\text{ReLU}(x) = \max(0, x)$$

Leaky Rectified Linear Unit (Leaky ReLU)

For negative input values, it returns a small linear function of the input instead of zero. Also helps mitigate the vanishing gradient problem.

$$\text{Leaky ReLU}(x) = \max(\alpha x, x), \quad \text{where } \alpha \text{ is a small constant}$$

Softmax Function

Often used in the output layer of neural networks for multi-class classification problems. It normalizes the output into a probability distribution over multiple classes.

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^N e^{x_j}}, \quad \text{for } i = 1, 2, \dots, N$$

3.2 Optimization Functions

Optimization functions iteratively adjust the weights and biases of model neurons to minimize the difference between the model's predicted results and the actual observed results. Some commonly used functions are stochastic gradient descent and Adam.

Stochastic Gradient Descent (SGD) A simple optimization algorithm that adjusts the network parameters to minimize the error function. It is used iteratively for each subset of data.

$$w_{t+1} = w_t - \alpha \nabla F(w_t)$$

Adam (Adaptive Moment Estimation) An optimization algorithm that combines the advantages of AdaGrad and RMSProp. It adjusts the learning rates for each parameter

independently.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla F(w_t) v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla F(w_t))^2 w_{t+1} = w_t - \frac{\alpha}{\sqrt{v_t} + \epsilon} m_t$$

3.3 Error Functions

Error functions measure how well the model predicts actual values. The goal is to minimize the value of the error function during the training process to make the model more accurate in its predictions. Some commonly used error functions are mean squared error and binary cross-entropy.

Mean Squared Error (MSE) Measures the average squared difference between actual and predicted values.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Binary Cross-Entropy (BCE) The error function for binary classification problems. It quantifies how well predicted probabilities match actual binary labels (0 or 1).

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

Furthermore, it is necessary to determine the learning rate. The learning rate determines the size of the step the algorithm takes in the negative direction of the gradient to minimize the loss function. If the learning rate is too large, it can lead to oscillations or large steps, making it difficult for the algorithm to converge or causing it to overshoot the minimum of the loss function. On the other hand, if the learning rate is too small, it may take many iterations to achieve convergence, and the algorithm may get stuck in a local minimum.

3.4 Graph Neural Networks

Graph neural networks are a type of neural network designed to work with data structured in the form of graphs. In GNNs graph nodes represent entities and edges represent

relationships between these entities. Graph neural networks have become popular in recent years due to their ability to learn and represent graph structures. They are suitable for graph analysis, including node classification, edge prediction, and graph classification. They have a variety of applications, most commonly in tasks involving relational data, such as social network analysis [10], recommendation systems [11], and text classification [10]. The idea of using GNNs to determine graph isomorphism lies in their ability to learn the structural properties of graphs.

3.5 Types of Graph Neural Networks

Graph neural networks (GNNs) operate by defining iterative update rules for node representations in a graph. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ represent a graph with nodes $\mathcal{V}$ and edges $\mathcal{E}$. At each iteration t , GNNs update the representation h_i^t of each node v_i based on information aggregated from its neighbors:

$$h_i^{t+1} = \text{AGGREGATE} \left(\{h_j^t : v_j \in \mathcal{N}(v_i)\} \right)$$

$$h_i^{t+1} = \text{UPDATE} (h_i^t, h_i^{t+1})$$

where $\mathcal{N}(v_i)$ denotes the neighborhood of node v_i , and AGGREGATE and UPDATE are learnable functions. This process enables nodes to incorporate both local and global graph context into their representations. GNNs extract features from the graph through parameterized transformations, refining these features iteratively to learn embeddings that encode the graph's structural and semantic properties. By optimizing a suitable loss function $\mathcal{L}$ using gradient-based techniques, GNNs learn to generate embeddings that enable downstream tasks such as node classification and link prediction. The expressive power and generalization ability of GNNs depend on the architecture, parameterization, and optimization strategy employed. Additionally, regularization techniques such as dropout and weight decay are often used to prevent overfitting and improve generalization.

Graph Convolutional Network (GCN) Graph Convolutional Network (GCN) [12] is one of the most commonly used types of graph neural networks. GCN uses local filtering to aggregate information from the neighborhoods of each node in the graph. For each node v_i with feature matrix H_i , the output of the GCN layer for that node is calculated as:

$$H'i = \sigma \left(\sum_{j \in N(i)} \frac{1}{c_{ij}} H_j W \right)$$

where $N(i)$ denotes the neighborhood of node v_i , W are filtering weights, σ is the activation function (e.g., ReLU), and c_{ij} is a normalization factor.

Graph Convolution (GraphConv) Graph Convolution is another type of graph neural network that uses convolutional operations on graphs. By using the adjacency matrix, GraphConv aggregates information from neighboring nodes to generate node representations. The convolutional output for each node v_i with feature matrix H_i is defined as:

$$H'i = \sigma \left(\sum_{j \in N(i)} H_j W \right)$$

where $N(i)$ denotes the neighborhood of node v_i , W are convolution weights, and σ is the activation function.

Graph Conditioned Convolution Graph (SAGE) The Graph Conditioned Convolution Graph (SAGE) [13] also uses convolution operations on graphs, but instead of using a fixed neighborhood structure, SAGE employs adaptive aggregation of information from different neighborhoods of each node. Mathematically, the SAGE layer for node v_i with feature matrix H_i is defined as:

$$H'_i = \sigma \left(\text{AGGREGATE} (H_j, \forall j \in N(i)) \cdot W \right)$$

where AGGREGATE represents the function for aggregating information from neighboring nodes, W are the convolution weights, and σ is the activation function.

Graph Attention Networks (GAT) Graph Attention Networks (GAT) [14] are a type of graph neural network that utilizes attention mechanisms to allow nodes to adaptively weight the information from their neighborhoods. The output of a GAT layer for node

v_i with feature matrix H_i is defined as:

$$H'i = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} H_j W \right)$$

where α_{ij} represents the attention weights between node v_i and its neighbors, W are the convolution weights, and σ is the activation function.

Chebyshev Convolution (ChebConv) Chebyshev Convolution (ChebConv) [15] is a type of graph neural network that uses polynomial interpolation to approximate convolution operations on graphs. The output of a ChebConv layer for node v_i with feature matrix H_i is defined as:

$$H'i = \sigma \left(\sum_{k=0}^{K-1} T_k(\tilde{L}) H W_k \right)$$

where $T_k(\tilde{L})$ represents Chebyshev polynomials of order k of the normalized Laplacian matrix $\tilde{L}$, W_k are the convolution weights, and σ is the activation function.

ARMA Convolution (ARMAConv) ARMA Convolution (ARMAConv) [16] is a type of graph neural network that combines autoregressive and moving average models for convolutional operations on graphs. The output of an ARMAConv layer for node v_i with feature matrix H_i is defined as:

$$H'_i = \sigma (\text{ARMA}(\tilde{L}) H W)$$

where $\text{ARMA}(\tilde{L})$ represents the ARMA model applied to the normalized Laplacian matrix $\tilde{L}$, W are the convolution weights, and σ is the activation function.

Spectral Graph Convolution (SGConv) Spectral Graph Convolutions (SGConv) [17] are a type of graph neural network that uses spectral graph theory to perform convolutional operations. The output of an SGConv layer for node v_i with feature matrix H_i is defined as:

$$H'i = \sigma \left(\sum_{k=0}^{K-1} \tilde{\Lambda}^k H W_k \right)$$

where $\tilde{\Lambda}$ is the diagonal matrix of eigenvalues of the Laplacian operator, W_k are the convolution weights, and σ is the activation function.

GIN Convolution (GINConv) Graph Isomorphism Network (GIN) Convolution (GINConv) [18] is a type of graph neural network that enhances the expressive power to capture graph structures. GINConv employs a learnable aggregation scheme that sums the features of neighboring nodes and applies a multi-layer perceptron (MLP) to the result. The output of a GINConv layer for node v_i with feature matrix H_i is defined as:

$$H'_i = \text{MLP} \left((1 + \epsilon) \cdot H_i + \sum_{j \in N(i)} H_j \right)$$

where $N(i)$ denotes the neighborhood of node v_i , ϵ is a learnable parameter, and MLP represents a multi-layer perceptron. This formulation allows GINConv to effectively capture different structural roles of nodes within the graph.

3.6 GNNs in the context of graph isomorphism problem, their power and limitations

In the context of graph isomorphism problem the effectiveness of GNNs relies on several key concepts and properties.

3.7 Representation of Graphs

The representation power of GNNs refers to their ability to encode the structural information present in graphs into distributed vector representations. Essentially, it's about transforming the complex relationships between nodes and edges into meaningful, high-dimensional embeddings.

3.8 Injectivity

Definition: Injectivity in the context of GNNs ensures that distinct elements or structures within a graph map to unique representations. Mathematically, a function $f : X \rightarrow Y$ is injective if each distinct element in the domain X corresponds to a unique element in the codomain Y . In GNNs, injectivity guarantees that different graphs or graph structures lead to distinct node embeddings. That means if we have two graphs with similar local structures but different global arrangements, an injective GNN would produce distinct node embeddings for nodes with similar local neighborhoods, allowing

for accurate differentiation between the two graphs.

3.9 Expressiveness

Definition: Expressiveness [19] in GNNs refers to the model’s capacity to capture and represent the structural differences between graphs accurately. It encompasses the GNN’s ability to discern subtle variations in graph topology, node attributes, and connectivity patterns. An expressive GNN can effectively capture the inherent complexity and diversity present in different graphs, enabling tasks such as graph classification and node prediction. An expressive GNN can accurately differentiate between graphs with similar node labels but distinct connectivity patterns, effectively capturing the underlying semantic differences between them.

The expressive power of GNNs remains a subject of debate and ongoing research. While some argue that the popular message passing GNNs can embed graphs as good as the 1-WL test can determine isomorphism [18]. Others say that GNNs are more powerful than the WL-1 test.[20].

3.10 Oversmoothing

Over-smoothing occurs in Graph Neural Networks (GNNs) when the iterative aggregation of neighborhood information leads to the convergence of node embeddings, resulting in the loss of discriminative power. Essentially, distinct nodes become indistinguishable in the embedding space due to excessive information propagation, hindering the GNN’s ability to accurately classify or distinguish between them.

4 Data generation

In the process of preparing data for training a graph classification model to discern between isomorphic and non-isomorphic graph pairs, several steps are involved. Firstly, the **relabel_nodes** function takes a graph and creates a deep copy, subsequently generating a random permutation of node labels and relabeling the nodes and their features accordingly. This process ensures that isomorphic pairs possess distinct node labels. Following this, the **generate_random_graph** function utilizes the Erdős-Rényi model to create random connected graphs with a specified number of nodes. This ensures the connectivity of generated graphs, a prerequisite for isomorphism checks. Once the graphs are generated, the **are_subgraphs_isomorphic** function determines if two graphs are isomorphic by converting their edge indices into NetworkX graphs and utilizing NetworkX's **is_isomorphic** function. After ensuring isomorphism, the **generate_dataset** function constructs a dataset comprising pairs of isomorphic and non-isomorphic graphs, assigning appropriate labels. Finally, the dataset undergoes shuffling to enhance the diversity of samples during model training.

A dataset of 400 pairs with 3, 4, 5, 6 or 7 nodes was generated. 80% of the data was used for training and validation, while 20% was used for testing. The number of non-isomorphic graphs with 3, 4, 5, 6 or 7 nodes is 1249[3].

The data is generated in the form presented in figures 4.1 and 4.3. Figures 4.2. and 4.4. represent the visualisation of these pairs. Each graph is described by its node features and edge connectivity. The node features, denoted by x , indicate the attributes associated with each node, where $[5, 16]$ signifies there are 6 nodes, each characterized by a feature vector of length 16. The edge index attribute specifies the connectivity of the graph, with the first dimension typically indicating the source nodes and the second dimension indicating the target nodes. For example, edge index= $[2, 4]$ suggests there are 4 edges

connecting the nodes. Additionally, the form includes labels associated with each graph pair, which will be used for tasks such as binary classification. Overall, this form serves as a structured representation of graph data, facilitating analysis and learning tasks.

```
Graph 1 Data:  
Data(x=[5, 16], edge_index=[2, 4])  
  
Graph 2 Data:  
Data(x=[5, 16], edge_index=[2, 4])  
  
Label: 1.0
```

Figure 4.1: Isomorphic pair

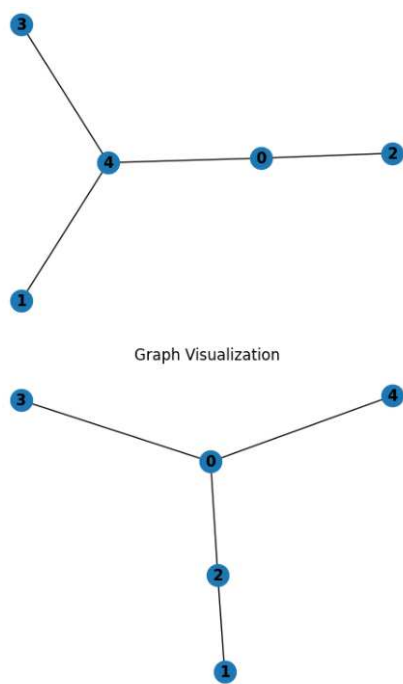


Figure 4.2: Isomorphic pair visualisation - example

```
Graph 1 Data:  
Data(x=[6, 16], edge_index=[2, 5])  
  
Graph 2 Data:  
Data(x=[6, 16], edge_index=[2, 6])  
  
Label: 0.0  
1.0
```

Figure 4.3: Non-Isomorphic pair

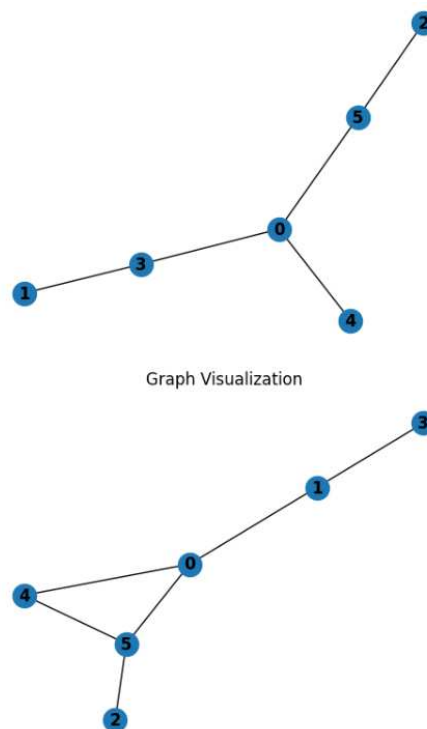


Figure 4.4: Non-Isomorphic pair visualisation - example

5 Implementation

Here we describe the model architecture used, the data generation process and the training and evaluation methodologies and functions used.

5.1 Models

A Siamese network is a type of neural network used for comparing and measuring the similarity between two input records [21]. It consists of two identical subsystems with shared weights. These subsystems generate feature vectors for each input, and then they are combined to make a decision about the similarity or dissimilarity between input pairs, i.e., in this project, a decision about the isomorphism of two graphs.

Model SiameseGNNWithDepth

The SiameseGNNWithDepth model is built with different types of Graph Neural Network (GNN) layers, including GCN (Graph Convolutional Network), SAGE (GraphSAGE), GAT (Graph Attention Network), GraphConv (Graph Convolution), ChebConv (Chebyshev Convolution), ARMAConv (Auto-Regressive Moving Average Convolution), SGConv (Spline-based Convolution) and GINConv (Graph Isomorphism Network). Each GNN type is equipped with two convolutional layers, with additional layers of the same type added based on the specified depth. Activation functions employed throughout the model include ReLU, Tanh, LeakyRelu and Sigmoid function, with the choice determined by the user. These layers and activation functions are compared to find the best combination. The models all have input layer dimension 16, hidden layer dimension 32 and output layer dimension 64.

5.2 Training and Evaluation

train(model, train_loader, optimizer, criterion)

One training epoch involves forward and backward passes, loss computation, backward propagation through the network to compute gradients, and an optimization step where new parameter values of the model are computed based on gradients and parameter values. Accuracy is monitored by comparing model predictions with actual labels.

validate(model, val_loader, criterion)

The model is evaluated on the validation set without weight updates, providing an unbiased estimate of its performance. Validation accuracy is computed for model comparison and selection.

test(model, test_loader, criterion)

The model is evaluated on the test set to assess generalization to new, unseen data. Test accuracy is computed for final model evaluation.

train_model(model, train_loader, val_loader, test_loader, optimizer, criterion, num_epochs)

This calls the train, validate, and test methods num_epochs times. It returns lists of training, validation, and test losses and accuracies.

6 Experiments

We tested the impact of different network architectures and other hyperparameters on the network's effectivity.

Training was done over 10 epochs. Eight different gnn-s as mentioned above were used, along with two different optimizers and loss functions, four activation functions and three learning rates, batch_sizes and depths for model training. One of the optimizers used was the Adam optimizer and the other was stochastic gradient descent (SGD). Regarding the loss functions, binary cross-entropy loss (BCELoss) and mean squared error loss (MSELoss) were included. Activation functions used were Relu, LeakyRelu, Sigmoid and Tanh. Additionally, learning rates of 0.0001, 0.001, 0.01 and 0.1, depths 2, 4, 6 and batch sizes 4, 16, 64 were compared.

6.1 Results

First, we show the top 10 results for each GNN layer type trained on a dataset of 400 graph pairs and compare how the results change depending on the hyperparameters (lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function). The Metric shown and used for comparison is test accuracy.

6.1.1 GCN

Table 6.1: Evaluation of the GCN layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.001	64	6	ReLU	MSELoss	SGD	0.7143
2.	0.0001	4	2	ReLU	BCELoss	Adam	0.7143
3.	0.01	16	6	LeakyReLU	BCELoss	SGD	0.7000
4.	0.1	4	6	ReLU	MSELoss	Adam	0.6786
5.	0.1	16	2	ReLU	BCELoss	Adam	0.6786
6.	0.01	4	4	ReLU	MSELoss	SGD	0.6786
7.	0.01	4	6	ReLU	MSELoss	Adam	0.6786
8.	0.01	64	4	ReLU	MSELoss	Adam	0.6786
9.	0.0001	16	6	ReLU	MSELoss	Adam	0.6786
10.	0.0001	64	6	ReLU	MSELoss	Adam	0.6786

GCN models perform best with activation function ReLu..

The best is the model with parameters: learning rate of 0.01, batch size of 64, depth of 6, ReLU activation, BCELoss as the loss function, and SGD optimizer, achieving an accuracy of 0.7143.

6.1.2 GraphConv

Table 6.2: Evaluation of the GraphConv layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.0001	16	6	ReLU	BCELoss	SGD	0.7500
2.	0.1	64	4	LeakyReLU	BCELoss	SGD	0.7250
3.	0.0001	64	6	ReLU	BCELoss	Adam	0.7143
4.	0.1	64	2	LeakyReLU	MSELoss	Adam	0.7000
5.	0.0001	64	4	ReLU	MSELoss	Adam	0.6786
6.	0.0001	64	4	ReLU	MSELoss	SGD	0.6786
7.	0.0001	4	4	LeakyReLU	BCELoss	Adam	0.6750
8.	0.001	64	6	LeakyReLU	MSELoss	Adam	0.6750
9.	0.0001	16	2	LeakyReLU	MSELoss	SGD	0.6500
10.	0.01	64	6	LeakyReLU	MSELoss	Adam	0.6500

Activation functions LeakyReLU and ReLu and larger batches like 64 seem to be the most effective for GraphConv.

The best is the model with parameters: learning rate of 0.0001, batch size of 16, depth of 6, ReLU activation, BCELoss as the loss function, and Adam optimizer, achieving an accuracy of 0.75.

6.1.3 SAGE

Table 6.3: Evaluation of the SAGE layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.1	4	2	LeakyReLU	BCELoss	SGD	0.7250
2.	0.0001	4	2	Tanh	MSELoss	Adam	0.7143
3.	0.1	4	4	ReLU	BCELoss	Adam	0.7143
4.	0.01	4	6	ReLU	MSELoss	SGD	0.7143
5.	0.01	64	6	ReLU	MSELoss	Adam	0.7143
6.	0.001	4	2	ReLU	BCELoss	SGD	0.7143
7.	0.001	64	6	ReLU	BCELoss	SGD	0.7143
8.	0.0001	4	4	LeakyReLU	BCELoss	SGD	0.7000
9.	0.001	4	6	Tanh	MSELoss	Adam	0.6786
10.	0.0001	16	2	Tanh	MSELoss	SGD	0.6786

Activation functions LeakyReLU and ReLU and smaller batches like 64 seem to be the most effective for GraphConv.

The best is the model with parameters: learning rate of 0.01, batch size of 16, depth of 2, LeakyReLU activation, BCELoss as the loss function, and Adam optimizer, with an accuracy of 0.7250.

6.1.4 GAT

Table 6.4: Evaluation of the GAT layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.1	4	4	Tanh	MSELoss	SGD	0.7857
2.	0.1	64	6	Tanh	BCELoss	SGD	0.7500
3.	0.01	64	4	Tanh	MSELoss	SGD	0.7500
4.	0.0001	4	6	ReLU	MSELoss	SGD	0.7500
5.	0.1	16	2	Tanh	MSELoss	SGD	0.7143
6.	0.1	16	2	ReLU	BCELoss	SGD	0.7143
7.	0.001	4	4	ReLU	BCELoss	Adam	0.7143
8.	0.01	64	2	Tanh	BCELoss	SGD	0.6786
9.	0.001	16	4	Tanh	MSELoss	SGD	0.6786
10.	0.001	16	6	Tanh	BCELoss	SGD	0.6786

GAT models achieve the highest accuracy among all tested models. It seems to perform best using the combination of Tanh activation function and SGD loss function.

The best is the model with parameters: learning rate of 0.1, batch size of 4, depth of 4,

Tanh activation, MSELoss as the loss function, and SGD optimizer, achieving an accuracy of 0.7857.

6.1.5 ChebConv

Table 6.5: Evaluation of the ChebConv layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.1	4	2	Tanh	BCELoss	Adam	0.7500
2.	0.0001	4	6	ReLU	BCELoss	Adam	0.7500
3.	0.01	4	4	Tanh	MSELoss	Adam	0.7143
4.	0.01	4	6	Tanh	BCELoss	Adam	0.7143
5.	0.1	64	4	ReLU	BCELoss	SGD	0.7143
6.	0.001	4	6	ReLU	MSELoss	Adam	0.7143
7.	0.0001	16	2	LeakyReLU	BCELoss	SGD	0.7000
8.	0.0001	64	2	LeakyReLU	MSELoss	SGD	0.7000
9.	0.1	64	6	Tanh	MSELoss	Adam	0.6786
10.	0.01	16	4	Tanh	MSELoss	Adam	0.6786

The best is the model with parameters: learning rate of 0.1, batch size of 4, depth of 2, Tanh activation, BCELoss as the loss function, and Adam optimizer, with an accuracy of 0.7500.

6.1.6 ARMAConv

Table 6.6: Evaluation of the ARMAConv layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.01	4	4	ReLU	MSELoss	Adam	0.7500
2.	0.001	4	6	Tanh	BCELoss	Adam	0.7250
3.	0.0001	64	2	ReLU	BCELoss	SGD	0.7000
4.	0.001	4	6	LeakyReLU	MSELoss	Adam	0.7000
5.	0.1	4	4	Tanh	MSELoss	Adam	0.7000
6.	0.01	16	4	Tanh	BCELoss	Adam	0.7000
7.	0.01	4	2	ReLU	MSELoss	Adam	0.6786
8.	0.01	16	4	ReLU	MSELoss	Adam	0.6786
9.	0.01	4	2	ReLU	BCELoss	SGD	0.6750
10.	0.01	64	6	ReLU	BCELoss	SGD	0.6750

ARMAConv performs well with ReLU activation. The best is the model with parameters: learning rate of 0.01, batch size of 4, depth of 4, ReLU activation, MSELoss as the loss function, and Adam optimizer, achieving an accuracy of 0.7500.

6.1.7 SGConv

Table 6.7: Evaluation of the SGConv layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.01	16	2	LeakyReLU	BCELoss	Adam	0.7250
2.	0.01	64	4	Tanh	MSELoss	Adam	0.7143
3.	0.01	64	6	Tanh	BCELoss	Adam	0.7143
4.	0.001	4	6	LeakyReLU	MSELoss	SGD	0.7000
5.	0.001	16	6	LeakyReLU	BCELoss	SGD	0.7000
6.	0.01	4	6	Tanh	BCELoss	Adam	0.6786
7.	0.01	16	4	Tanh	MSELoss	SGD	0.6786
8.	0.001	16	6	Tanh	BCELoss	Adam	0.6786
9.	0.01	64	6	LeakyReLU	MSELoss	Adam	0.6750
10.	0.1	64	2	LeakyReLU	MSELoss	SGD	0.6500

Activation functions LeakyReLU and Tanh seem to be the most effective for SGConv.

The best is the model with parameters: learning rate of 0.01, batch size of 4, depth of 4 ,ReLU activation, BCELoss as the loss function, and Adam optimizer, achieving an accuracy of 0.7250.

6.1.8 GINConv

Table 6.8: Evaluation of the GINConv layer networks, depending on the layer type, lr - learning rate, b - batch size, n - number of GNN layers, optimiser, activation and loss function. Test accuracy is shown.

	lr	b	d	activation	loss	optimizer	accuracy
1.	0.001	16	6	ReLU	MSELoss	Adam	0.7000
2.	0.1	16	2	LeakyReLU	MSELoss	Adam	0.6750
3.	0.1	64	6	LeakyReLU	BCELoss	Adam	0.6750
4.	0.001	16	6	LeakyReLU	MSELoss	Adam	0.6750
5.	0.01	64	4	ReLU	MSELoss	Adam	0.6500
6.	0.001	16	2	ReLU	MSELoss	SGD	0.6500
7.	1e-06	16	4	ReLU	BCELoss	SGD	0.6500
8.	1e-06	64	6	ReLU	MSELoss	Adam	0.6500
9.	0.001	16	2	LeakyReLU	BCELoss	SGD	0.6500
10.	0.001	64	2	LeakyReLU	BCELoss	Adam	0.6500

GINConv models perform best in bigger batches, such as 16 and 64, activation functions ReLU and LeakyRelu and optimizer Adam. GINConv layers were expected to perform better due to their theoretical advantage in recognizing non-isomorphic graphs. However, they underperformed in this specific task, indicating that their theoretical strengths did not translate into better practical performance in this context.

The best is the model with parameters: learning rate of 0.001, batch size of 16, depth of 6, ReLU activation, MSELoss as the loss function, and Adam optimizer, resulting in an accuracy of 0.7000.

6.2 Comparison

Network layer type GAT performs best on this task, followed by ARMAConv, ChebConv, SGConv, SAGE and GraphConv. GCN and GINConv generally demonstrated lower performance. We also observe that in 7 out of 10 best models, the false positive rate is higher than the false negative rate.

Table 6.9: Comparison of best models from previous chapters, with additional metrics FP - false positive rate and FN - false negative rate

	lr	b	d	activation	loss	optimizer	accuracy	FP	FN	GNN Type
1.	0.1	4	4	Tanh	MSELoss	SGD	0.7857	0.1786	0.0357	GAT
2.	0.1	16	4	ReLU	BCELoss	Adam	0.7857	0.0000	0.2143	SGConv
3.	0.1	64	6	Tanh	BCELoss	SGD	0.7500	0.2143	0.0357	GAT
4.	0.01	64	4	Tanh	MSELoss	SGD	0.7500	0.0357	0.2143	GAT
5.	0.1	4	2	Tanh	BCELoss	Adam	0.7500	0.1429	0.1071	ChebConv
6.	0.01	4	4	ReLU	MSELoss	Adam	0.7500	0.2143	0.0357	ARMAConv
7.	0.0001	16	6	ReLU	BCELoss	SGD	0.7500	0.2500	0.0000	GraphConv
8.	0.0001	4	6	ReLU	BCELoss	Adam	0.7500	0.0357	0.2143	ChebConv
9.	0.0001	4	6	ReLU	MSELoss	SGD	0.7500	0.2500	0.0000	GAT
10.	0.001	4	6	Tanh	BCELoss	Adam	0.7250	0.1500	0.1250	ARMAConv

In examining the loss and accuracy graphs for the models with the highest test accuracy, the biggest issue is the fact the validation loss increases from the start across all models. Despite implementing careful validation set construction, and extensive hyperparameter tuning, this pattern persists. This suggests that the models may be inherently overfitting due to their complexity relative to the dataset. Thus, the observed behavior likely reflects issues related to model complexity and data characteristics. This insight leads us to conclude that, given the level of complexity in the chosen approach to tackle this task, the challenges in accurately determining graph isomorphism primarily stem from inherent limitations within GNNs.

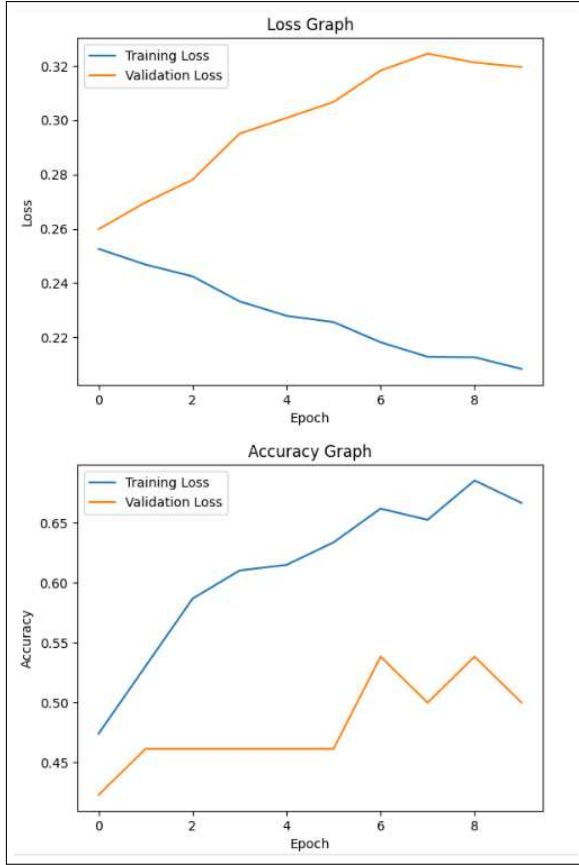


Figure 6.1: Train and validation loss and accuracy graphs for the 1. best model with GAT layer type and parameters $lr = 0.1$, $b = 4$, $d = 4$, activation function = tanh, loss function = MSELoss and optimizer = SGD

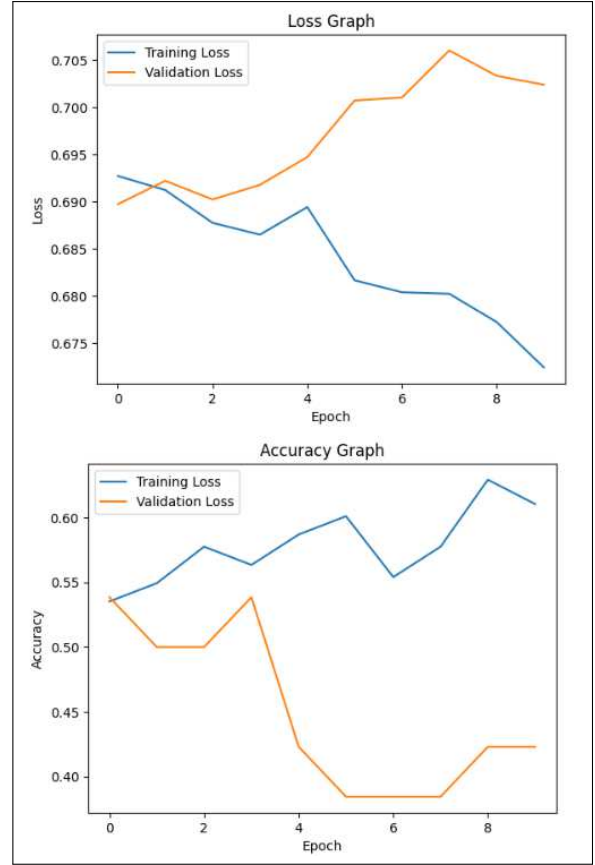


Figure 6.2: Train and validation loss and accuracy graphs for the 2. best model with SG-Conv layer type and parameters $lr = 0.1$, $b = 16$, $d = 4$, activation function = ReLu, loss function = BCELoss and optimizer = Adam

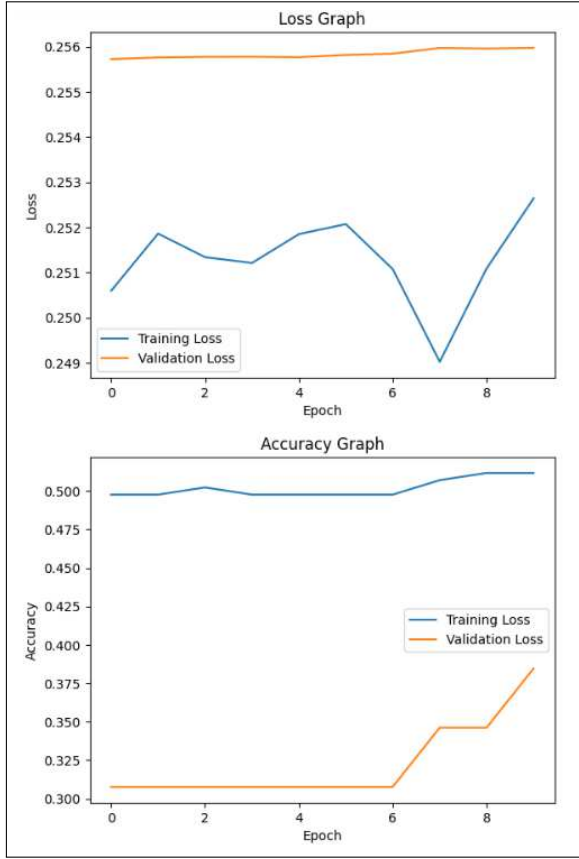


Figure 6.3: Train and validation loss and accuracy graphs for the 3. best model with GAT layer type and parameters $lr = 0.01$, $b = 64$, $d = 4$, activation function = tanh, loss function = MSELoss and optimizer = SGD

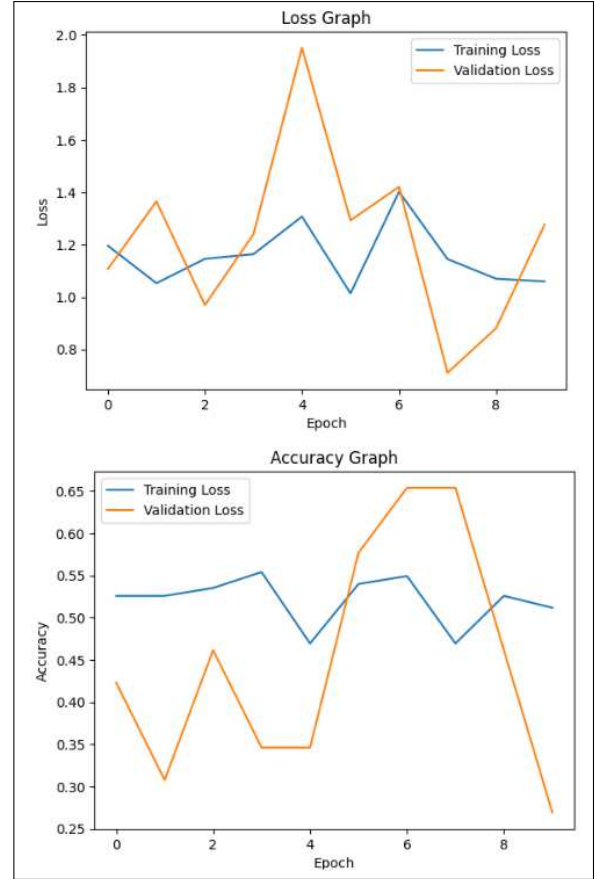


Figure 6.4: Train and validation loss and accuracy graphs for the 4. best model with Cheb-Conv layer type and parameters $lr = 0.1$, $b = 4$, $d = 2$, activation function = tanh, loss function = BCELoss and optimizer = Adam

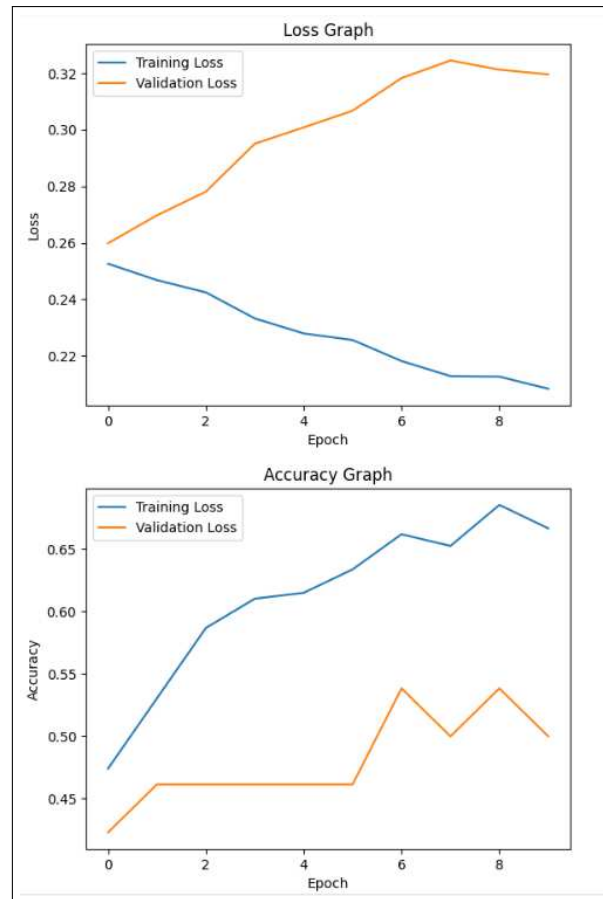


Figure 6.5: Train and validation loss and accuracy graphs for the 5. best model with GAT layer type and parameters $lr = 0.01$, $b = 4$, $d = 4$, activation function = ReLu, loss function = MSELoss and optimizer = Adam

7 Discussion

The results from our experiments revealed consistent poor performance across all tested GNN architectures and parameter configurations. Despite variations in hyperparameters and network architectures, the ability of GNNs to accurately determine graph isomorphism remained elusive.

One plausible explanation for this limitation is that GNNs represent graphs in a way that they can't differentiate between similar but non-isomorphic graphs. GNNs represent graphs by aggregating information from local and global structures, but they may struggle to capture subtle structural differences that are crucial for identifying isomorphism. The high false positive rates observed in our experiments support this notion, indicating that GNNs may misclassify non-isomorphic graphs as isomorphic due to their inability to discern these subtle differences.

8 Conclusion

In this thesis, the application of deep graph neural networks to the task of testing the isomorphism of two graphs. The development and training of a deep graph neural network on a generated dataset have provided findings regarding the impact of different network architectures and parameters on prediction accuracy.

In conclusion, trying to use deep graph neural networks (GNNs) for graph isomorphism testing has highlighted significant challenges in accurately distinguishing between isomorphic and non-isomorphic graphs. Despite testing various GNN architectures and parameters, consistent limitations were found, probably due to their inability to capture subtle structural differences between graphs.

These findings underscore the complexity of the graph isomorphism problem and emphasize the need for further research. Future research could focus on developing specialized GNN architectures tailored specifically for graph isomorphism testing. Additionally, investigating hybrid models that combine GNNs with other machine learning approaches, may offer new insights and improve performance in this challenging task.

Overall, this study contributes to the ongoing dialogue surrounding the intersection of graph theory and deep learning, offering more understanding on the capabilities of deep learning techniques in graph-related tasks.

9 Code

For the purpose of this final project, the Google Colab platform and the PyTorch tool were used. The Google Colab platform allows for direct execution of someone's existing code from the GitHub platform or from Jupyter Notebooks. The Google Colab notebook provides simple and free access to a GPU, which speeds up the processing of large datasets and provides clarity. A special library, PyTorch Geometric [22], specifically designed to work with graph data and graph neural networks, was used.

Before running anything in the Google Colab, it is necessary to click on -> Runtime -> Change runtime type -> GPU.

Link to the notebook:

https://colab.research.google.com/drive/1FA0xmmX_kH8QQxqWivNdWlgU4VhZfce8?usp=sharing

All results were saved in a JSON file.

Link to the results:

https://drive.google.com/file/d/12YnG2J8WrTwU1HR8oq9q3Dkz0cNmL2RB/view?usp=drive_link

References

- [1] R. J. Wilson, *Introduction to Graph Theory*, 4th ed. Addison Wesley Longman Limited, 1996.
- [2] A. Tucker, *Applied Combinatorics*, 6th ed. Hoboken, NJ: John Wiley & Sons, Inc., 2012.
- [3] A. Nakić and I. Vesel, “Problem klasifikacije grafova,” *ACTA MATHEMATICA SPALATENSIA Series didactica*, vol. 3, pp. 87–97, 2020.
- [4] L. Babai, “Graph isomorphism in quasipolynomial time,” *CoRR*, vol. abs/1512.03547, p. 684–697, 2015.
- [5] S. Kiefer, “Power and limits of the weisfeiler-leman algorithm,” Ph.D. dissertation, RWTH Aachen University, 2020.
- [6] B. D. McKay and A. Piperno, “Practical graph isomorphism, ii,” *Journal of Symbolic Computation*, vol. 60, pp. 94–112, 2014.
- [7] J. R. Ullmann, “An algorithm for subgraph isomorphism,” *J. ACM*, vol. 23, no. 1, p. 31–42, jan 1976.
- [8] Ronnasayd, “Deep learning book,” GitHub Repository.
- [9] A. B. Downey, *Think Complexity*. Needham, Massachusetts: Green Tea Press, 2003.
- [10] Z. Wang, T. Chen, J. Ren, W. Yu, H. Cheng, and L. Lin, “Deep reasoning with knowledge graph for social relationship understanding,” vol. 27, 2018, pp. 1021–1028.

- [11] Q. Wu, H. Zhang, X. Gao, P. He, P. Weng, H. Gao, and G. Chen, “Dual graph attention networks for deep latent representation of multifaceted social effects in recommender systems.” *ACM*, May 2019, pp. 2091–2102.
- [12] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24-26, 2017, Conference Track Proceedings*.
- [13] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” *Curran Associates Inc.*, p. 1025–1035, 2017.
- [14] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph Attention Networks,” *International Conference on Learning Representations*, 2018.
- [15] M. Defferrard, X. Bresson, and P. Vandergheynst, “Convolutional neural networks on graphs with fast localized spectral filtering,” ser. NIPS’16. Red Hook, NY, USA: Curran Associates Inc., 2016, p. 3844–3852.
- [16] F. M. Bianchi, D. Grattarola, L. Livi, and C. Alippi, “Graph neural networks with convolutional arma filters,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3496–3507, 2022.
- [17] Y. Li, T. Cai, Y. Zhang, D. Chen, and D. Dey, “What makes convolutional models great on long sequence modeling?” *The Eleventh International Conference on Learning Representations*, Oct 2022.
- [18] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*.
- [19] B. Zhang, C. Fan, S. Liu, K. Huang, X. Zhao, J. Huang, and Z. Liu, “The expressive power of graph neural networks: A survey,” *CoRR*, vol. abs/2308.08235, 2023.
- [20] C. I. Kanatsoulis and A. Ribeiro, “Graph neural networks are more powerful than we think,” pp. 7550–7554, *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024.

- [21] S. Stefanovic. (2021, July 11) #019 siamese network in pytorch with application to face similarity.
- [22] Pytorch geometric documentation. PyTorch Geometric.

Testiranje izomorfizma dvaju grafova pomoću dubokih neuronskih mreža grafova

Sažetak

Ovaj rad istražuje primjenu dubokih neuronskih mreža na problem testiranja izomorfizma grafova, fundamentalnog izazova u teoriji grafova s raznolikim primjenama. Grafovi su matematičke strukture sastavljene od vrhova i bridova, a određivanje jesu li dva grafa izomorfna je računalno složen zadatak. Duboke neuronske mreže grafova pružaju pristup rješavanju tog izazova učenjem složenih reprezentacija struktura grafova. U ovom istraživanju, razvijena je i trenirana duboka neuronska mreža grafova na skupu generiranih podataka kako bi predviđela izomorfizam grafova. Kroz eksperimentiranje i analize, istražen je utjecaj različitih arhitektura mreža i parametara na točnost predikcije. Zaključak je da popularne graf neuronske mreže nemaju dobru sposobnost determinacije izomorfizma, potencijalno jer ne razlikuju dovoljno dobro grafove koji su slučajni ali nisu izomorfni.

Ključne riječi: teorija grafova, izomorfizam grafova, duboke neuronske mreže, neuronske mreže grafova

Testing the Isomorphism of Two Graphs Using Deep Graph Neural Networks

Abstract

This thesis investigates the application of deep graph neural networks to the problem of graph isomorphism testing, a fundamental challenge in graph theory with wide-ranging applications. Graphs are essential mathematical structures composed of vertices and edges, and determining whether two graphs are isomorphic is a computationally complex task. Deep graph neural networks offer a promising approach to address this challenge by learning intricate representations of graph structures. In this study, a deep graph neural network is developed and trained on a generated dataset to predict graph isomorphism. Through experimentation and analysis, the influence of different network architectures and parameters on prediction accuracy is examined. The conclusion is that popular Graph Neural Networks lack the

ability to effectively determine isomorphism. This may be because they do not sufficiently distinguish between graphs that are similar but not isomorphic.

Keywords: graph theory, graph isomorphism, deep neural networks, graph neural networks